Linux2 Oracle Cloud Hosting Runbook

How Docs to Markdown was deployed on Sean's Ubuntu host, and how to operate it.

This is the public-hosting path. Local Windows testers still use LAUNCH.bat on http://127.0.0.1:8000/. Do not mix the two.

Privacy. *Sean is named as the VM owner. This PDF does not contain host IPs, SSH account names, emails, or public keys. Use the placeholders* `<HOST_IP>`, `<SSH_USER>`, *and* `<REPO_URL>`. *Ask Sean for the current values. Never commit those values to Git.*

1. What this is

Item	Value
App	Docs to Markdown (this repository)
Host nickname	linux2
Cloud	Oracle Cloud Infrastructure (OCI) compute
OS	Ubuntu 24.04 LTS, x86_64
Public IPv4	<HOST_IP> — ask Sean; never publish the real address
SSH user	<SSH_USER>
SSH	ssh <SSH_USER>@<HOST_IP>
App URL	http://<HOST_IP>:8000/
Converter	http://<HOST_IP>:8000/converter
Health	http://<HOST_IP>:8000/health
Container	docs-to-markdown
Image	docs-to-markdown:latest
Published port	TCP 8000 → container 8000
Repo on host	~/MKDocs-File-Conversation-to-MD

The library, themed converter, batch page, and APIs all share **one port**. There is no separate 8001.

2. Who does what

Sean provided the VM and handles cloud firewall / host admin. Daniel operates inside the VM as a normal user with Docker.

Person	Can do	Cannot do
Sean	VM access, add SSH public keys, open/close cloud firewall ports, host sudo	—
Daniel	SSH in, use Docker without sudo, clone the repo, build/run/stop the app container	sudo, change cloud firewall rules from the VM, install host packages

Confirmed on first login:

- Login user is in the `docker` group.
- `docker ps` works without sudo.

- sudo requires a password this account does not have.

If a step needs sudo or an OCI console click, that is Sean’s job. Everything that is Docker or files in the home directory is Daniel’s job.

3. What we actually did

Real sequence from 21 August 2026, not a hypothetical.

1. **SSH key on the Windows PC.** Generated a local ed25519 key pair. Never send the private key. Only the .pub file was given to Sean.

2. **Sean installed that public key** on linux2 in the login user’s authorized_keys file.

3. **First SSH test** from Windows succeeded. Host key was added to known_hosts.

4. **Inspected the box.** Docker Engine 29.x with Compose and Buildx. About 36 GB free disk. About 1 GB RAM and no swap. Host firewall was not visible to this user; the real gate is the cloud security list / NSG. Only TCP 22 was listening publicly at that moment.

5. **Asked Sean to open TCP 8000** on the cloud security list / NSG (ingress). Optional later: 80 and 443 for a reverse proxy. SSH 22 was already open.

6. **Cloned this repository** onto the box into ~/MKDocs-File-Conversation-to-MD (branch main).

7. **The GitHub tree had no Dockerfile.** A production Dockerfile and .dockerignore were written and copied to the box (section 6).

8. **Built the image on the box** (~5 minutes): docker build -t docs-to-markdown:latest .

9. **Ran the container** with --restart unless-stopped and -p 8000:8000. First ~40 seconds were MkDocs site build during FastAPI startup. Then Uvicorn listened on 0.0.0.0:8000, Docker health was healthy, and GET /health returned {"status":"ok"}.

10. **Verified from outside the VM** that TCP 8000 was open: public GET /health returned HTTP 200.

11. **End-to-end functional test** against the live container (section 9).

4. Access from Windows (repeatable)

OpenSSH is already on the PC (Windows OpenSSH).

4.1 One-time key

```
New-Item -ItemType Directory -Path "$env:USERPROFILE\.ssh" -Force
ssh-keygen -t ed25519 -C "docs-to-markdown" -f "$env:USERPROFILE\.ssh\id_ed25519" -N '""'
Get-Content "$env:USERPROFILE\.ssh\id_ed25519.pub"
```

Send Sean **only** the .pub line. He appends it to authorized_keys for <SSH_USER>.

Optional, so you do not pass -i every time:

```
Get-Service ssh-agent | Set-Service -StartupType Manual
Start-Service ssh-agent
ssh-add "$env:USERPROFILE\.ssh\id_ed25519"
```

4.2 Login

```
ssh -i $env:USERPROFILE\.ssh\id_ed25519 <SSH_USER>@<HOST_IP>
```

Or open a visible window:

```
Start-Process wt -ArgumentList "ssh","-i","$env:USERPROFILE\.ssh\id_ed25519","<SSH_USER>@<HOST_IP>"
```

BatchMode after the host key is known:

```
ssh -o BatchMode=yes -i $env:USERPROFILE\.ssh\id_ed25519 <SSH_USER>@<HOST_IP> "whoami; hostname"
```

Expected: <SSH_USER> / linux2.

4.3 Copy a file to the box

```
scp -i $env:USERPROFILE\.ssh\id_ed25519 .\Dockerfile <SSH_USER>@<HOST_IP>:~/MKDocs-File-Conversation-to-MD/
```

5. Ports Sean must keep open

The VM can bind 0.0.0.0:8000 and still be unreachable if OCI blocks it. Ask Sean to set Ingress on the instance VCN security list or network security group.

Direction	Protocol	Port	Source	Why
Ingress	TCP	22	tester IPs, or 0.0.0.0/0 if already used	SSH (already working)
Ingress	TCP	8000	0.0.0.0/0 or a tight allow-list	Docs to Markdown HTTP
Ingress	TCP	80	optional	HTTP via a reverse proxy later
Ingress	TCP	443	optional	HTTPS via a reverse proxy later

Do not need 8001. Do not open Docker's random high ports.

Copy-paste for Sean:

```
Need ingress TCP 8000 on linux2
(source 0.0.0.0/0, or lock to tester IPs).
22 is already fine.
Optional later: 80 and 443 if we put it behind a reverse proxy.
```

On the VM, confirm the container is listening:

```
ss -tlnp | grep 8000
```

From a laptop that is not the VM:

```
Invoke-WebRequest http://<HOST_IP>:8000/health -UseBasicParsing
```

If the container is healthy but this times out, the OCI NSG is still closed.

6. Dockerfile used on linux2

This repository originally had no Dockerfile. The image on linux2 was built from the files below, placed in the repo root on the box.

Dockerfile

```
FROM python:3.12-slim-bookworm

COPY --from=ghcr.io/astral-sh/uv:0.8.22 /uv /usr/local/bin/uv

RUN apt-get update \
    && apt-get install -y --no-install-recommends libgomp1 \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

ENV UV_COMPILE_BYTECODE=1 \
    UV_LINK_MODE=copy \
    UV_PYTHON_DOWNLOADS=0 \
```

```
    PATH="/app/.venv/bin:$PATH" \
    HOME=/tmp \
    PYTHONUNBUFFERED=1

COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev --extra site --no-install-project --no-editable

COPY src ./src
COPY mkdocs-site ./mkdocs-site
RUN uv sync --frozen --no-dev --extra site --no-editable

RUN useradd --create-home --uid 1001 appuser \
    && chown -R appuser:appuser /app
USER appuser

EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --start-period=40s \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health')"

CMD ["uvicorn", "docs_to_markdown.api:app", "--app-dir", "src", "--host", "0.0.0.0", "--port", "8000"]
```

Why these choices:

- **Python 3.12 slim** matches requires-python >= 3.12.
- **uv locked sync** (--frozen) matches uv.lock. --extra site installs MkDocs so the library can build at startup. --no-dev keeps pytest out of production.
- **libgomp1** is required by ONNX Runtime / RapidOCR.
- **src + mkdocs-site** must be siblings. mkdocs_publish.py resolves the site as two parents up from the package, then / mkdocs-site.
- **Listen 0.0.0.0:8000**, not 127.0.0.1. Inside Docker, localhost is only the container.
- **Non-root UID 1001.**
- **Healthcheck** hits /health, which returns {"status":"ok"}.
- **start-period 40s** because startup runs build_mkdocs_site() and can take half a minute on this 1 GB box.

.dockerignore

```
.git
.github
.venv
**/__pycache__
**/*.pyc
tests
docs
sample-docs
site
mkdocs-site/site
INSTALL_AND_LAUNCH.ps1
LAUNCH.bat
AGENTS.md
```

sample-docs stays out of the image. Library content that is baked in comes from mkdocs-site/docs/. Built HTML is generated at container start into mkdocs-site/site.

7. Build on the box

```
cd ~/MKDocs-File-Conversation-to-MD
git pull --ff-only origin main
docker build -t docs-to-markdown:latest .
docker images docs-to-markdown
```

First build pulled python:3.12-slim-bookworm and the official uv image, then installed 67 locked packages (FastAPI, MarkItDown, pdfplumber, RapidOCR, OpenCV, MkDocs, …). Observed wall time: about 5 minutes.

The box has no swap and about 1 GB RAM. Do not run a second heavy build while the app is converting a large PDF. If the build is OOM-killed, ask Sean for more RAM or a small swap file (that needs sudo).

8. Run / stop / restart

Start (first time, or after docker rm)

```
docker run -d \
  --name docs-to-markdown \
  --restart unless-stopped \
  -p 8000:8000 \
  docs-to-markdown:latest
```

--restart unless-stopped brings it back after a VM reboot unless you explicitly stopped it.

Day-2 commands

```
docker ps --filter name=docs-to-markdown
docker logs -f --tail 100 docs-to-markdown
docker restart docs-to-markdown
docker stop docs-to-markdown
docker start docs-to-markdown
```

Replace with a newly built image

```
cd ~/MKDocs-File-Conversation-to-MD
git pull --ff-only origin main
docker build -t docs-to-markdown:latest .
docker rm -f docs-to-markdown
docker run -d --name docs-to-markdown --restart unless-stopped -p 8000:8000 docs-to-markdown:latest
```

Wait until logs show Application startup complete and docker ps says (healthy) before testing.

Published files inside the container are lost on docker rm unless you add a volume. The Git clone on disk is separate from the running container filesystem. To persist library uploads across rebuilds (not used on the first deploy):

```
docker run -d \
  --name docs-to-markdown \
  --restart unless-stopped \
  -p 8000:8000 \
  -v ~/MKDocs-File-Conversation-to-MD/mkdocs-site:/app/mkdocs-site \
  docs-to-markdown:latest
```

Only add that volume if you intend the container to write into the git working tree.

9. How we proved it works

All of the following passed against the live container on 21 August 2026.

Check	Result
docker ps	Up, port 0.0.0.0:8000→8000/tcp, health healthy
GET /health (on-box and public)	HTTP 200 {"status":"ok"}
GET /	HTTP 200 HTML — MkDocs library home
GET /converter	HTTP 200 HTML — themed converter
GET /batch	HTTP 200 HTML
GET /app/converter	HTTP 200 HTML — standalone converter

Check	Result
POST /api/convert sample PDF	HTTP 200 in 9.1 s, Markdown with extracted page images
POST /api/convert matching DOCX	HTTP 200 in 1.2 s, headings and tables present
POST /api/render	HTTP 200 HTML preview
POST /api/convert a .txt file	HTTP 415 Only DOCX and PDF files are supported

On-box convert example:

```
PDF=~/MKDocs-File-Conversation-to-MD/sample-docs/it-service-desk/<sample>.pdf
curl -sS -F "file=@$PDF" http://127.0.0.1:8000/api/convert
```

Browser smoke: open http://<HOST_IP>:8000/converter, drop a .docx or .pdf, Convert, confirm Markdown appears.

10. URLs and routes

Same process, same port. Replace <HOST_IP> with the value Sean gives you.

URL	What you get
http://<HOST_IP>:8000/	Document library home
http://<HOST_IP>:8000/converter	Themed converter (upload / preview / publish)
http://<HOST_IP>:8000/app/converter	Legacy standalone converter
http://<HOST_IP>:8000/batch	Batch ZIP converter
http://<HOST_IP>:8000/health	Liveness JSON
http://<HOST_IP>:8000/api/convert	POST multipart file
http://<HOST_IP>:8000/api/render	POST JSON markdown preview
http://<HOST_IP>:8000/api/publish	POST publish into the MkDocs library
http://<HOST_IP>:8000/mkdocs/	Built site mount

11. Host limits and gotchas

- **~1 GB RAM, 0 swap.** The running app sat around 190 MiB after startup. A large PDF convert plus a docker build at the same time can OOM the VM.
- **Startup is slow.** Uvicorn logs Waiting for application startup until MkDocs finishes. curl during that window can see connection reset. Wait for Application startup complete.
- **No sudo.** You cannot apt install, enable UFW, or add swap yourself. Ask Sean.
- **Public service.** Anyone who can reach :8000 can convert files and, if they find publish, write into the container’s library. Treat it as a tester box. Do not upload credentials or customer PII.
- The corporate Windows proxy described in AGENTS.md does not apply on linux2. uv / Docker Hub / GitHub HTTPS from the VM worked without extra cert flags.

12. Troubleshooting

Permission denied (publickey)

Sean does not have your current .pub in authorized_keys, or you are using a different private key. Print the pub, send it again, SSH with -i pointing at the matching private key.

SSH works, browser times out

Container not running, or OCI NSG missing TCP 8000.

```
docker ps --filter name=docs-to-markdown
ss -tlnp | grep 8000
curl -sS http://127.0.0.1:8000/health
```

If on-box curl works and public curl does not, it is the Oracle security list.

Container restarts / unhealthy

```
docker logs --tail 200 docs-to-markdown
docker inspect docs-to-markdown
free -h
```

If OOMKilled is true, ask Sean for RAM or swap.

Only DOCX and PDF files are supported

Expected for any other extension (HTTP 415).

Conversion 422

Empty file, or the engine could not parse that document. Retry with a sample under sample-docs/.

Stale UI after a rebuild

Hard-refresh the browser. Confirm you hit :8000 on the host, not an old local LAUNCH.bat on 127.0.0.1:8000.

Need to start over

```
docker rm -f docs-to-markdown
cd ~/MKDocs-File-Conversation-to-MD
docker build -t docs-to-markdown:latest .
docker run -d --name docs-to-markdown --restart unless-stopped -p 8000:8000 docs-to-markdown:latest
```

13. Security notes

- Share public keys only (*.pub). The file id_ed25519 without .pub is the private key.
- Do not commit host IPs, account names, emails, SSH keys, OCI API keys, or real uploaded documents to Git.
- Prefer locking NSG source IPs to testers instead of 0.0.0.0/0 once the app is no longer a wide demo.
- The login account cannot sudo; that is intentional isolation on the shared VM.

14. Quick reference

```
# SSH
ssh <SSH_USER>@<HOST_IP>

# Status
docker ps --filter name=docs-to-markdown
curl -sS http://127.0.0.1:8000/health

# Logs
docker logs -f --tail 100 docs-to-markdown

# Rebuild and replace
cd ~/MKDocs-File-Conversation-to-MD
git pull --ff-only origin main
docker build -t docs-to-markdown:latest .
docker rm -f docs-to-markdown
```

```
docker run -d --name docs-to-markdown --restart unless-stopped -p 8000:8000 docs-to-markdown:latest

# Public check
http://<HOST_IP>:8000/health
http://<HOST_IP>:8000/converter
```